

# Cover Letter for TOSEM Submission

*TyFlow: A Type-Aware Approach to Neural Code Models*

Manuscript TOSEM-2026-0076 • Major Revision, decision of June 16, 2026

---

We thank the reviewers and the editor for their valuable comments. We have revised our paper accordingly. Every reviewer comment is quoted below in italics, followed by our response and by a summary of the changes we made in the paper; section and page numbers refer to the revised manuscript (45 pages).

**Summary of changes.** The revision adds two Java benchmarks (HumanEval-Java and TransCoder-GFG) that enlarge the Java evaluation to 186 tasks, a statistical analysis with 95% intervals and exact paired tests (App. D), a task-level failure taxonomy (App. C), runtime and pruning measurements of the constrained decoder (RQ2), a comparison with three decoder-only models of 7B–30B parameters (Table 3), and a comparison with four reliability-oriented generation methods (RQ3, Table 5). It also adds a Limitations subsection (Sec. 6.3) on the encoder-decoder dependence, dynamically typed languages, and type-system expressiveness.

## Response to the Editor

---

### Editor’s comment.

*All the reviewers agree that the paper investigates an important topic. However, reviewers also point out major concerns that need to be addressed and responded to, hence my recommendation for a Major Revision. In the following, I have provided some of the critical concerns raised by the reviewers.*

- *Missing failure analysis*
- *Limited evaluation benchmarks*
- *Missing discussion on scalability*
- *Missing comparison with larger and more recent LLMs*
- *Missing comparison with state-of-the-art techniques*
- *Missing analysis of the system’s cost*

### Response.

Thanks. Each of the six concerns is addressed with new experiments or new analyses, as summarized below:

- **Failure analysis.** Every test task of the three Java benchmarks is classified into four categories in App. C (p. 42–43, Table 8), with two representative cases; the dominant

residual mode of TyFlow is well-typed but semantically wrong code. Detailed response: Comment 1.6.

- **Benchmarks.** HumanEval-Java (146/16) and TransCoder-GFG (414/103) are added, so the Java evaluation spans 186 test tasks from three independent sources, analyzed jointly in App. D (p. 44). Detailed responses: Comments 1.1, 2.1.
- **Scalability.** RQ2 measures the pruning strength of each component, the beam-exhaustion behavior, and the per-step and per-task runtime (up to  $2.74\times$  the unconstrained base); scaling to richer type systems is discussed in the Limitations subsection. Detailed responses: Comments 1.5, 2.2.
- **Larger and more recent LLMs.** Zero- and few-shot pass@1 results for MiMo-7B, Qwen3-14B, and Qwen3-30B-A3B on all four benchmarks are added (Table 3, p. 26): on Java they outperform the fine-tuned 2B models, on SuFu none solves a single task zero-shot. Detailed responses: Comments 1.3, 2.3.
- **State-of-the-art techniques.** RQ3 is rewritten as a five-method comparison on MBJP (rejection sampling, SynCode, Repilot, iterative compiler repair) on one leak-clean frozen checkpoint; none of the four external methods significantly improves functional correctness, while TyFlow-2B is the best of all compared methods. Detailed responses: Comments 3.1, 3.2.
- **System cost.** The runtime analysis is reported in RQ2 together with the component ablation it belongs to: per-step and per-task wall time of the same 220M model in the four configurations, with an explanation of where each cost comes from and why syntactic pruning even *lowers* the per-task total. For the compared external methods, the RQ3 setup states each method’s computational budget. Detailed responses: Comments 2.2, 3.3.

The individual reviewer comments, with the corresponding responses, follow.

## Response to Reviewer 1

We thank the reviewer for the positive assessment of the formalization, its generality over grammar-based approaches, and the ablation study.

### Comment 1.1.

*The SuFu test set has only ~58 tasks (20% of 290), and the Java test set has only ~61 tasks (10% of 608). These are very small test sets, and the results may not be statistically robust. The authors should report confidence intervals or conduct significance tests. A difference of 1–2 tasks passing or failing could shift pass@1 by several percentage points.*

[Weaknesses:] *Small test sets raise concerns about statistical reliability. [ . . . ] Missing failure*

*analysis and confidence intervals.*

### Response.

Thanks for this comment. We addressed the concern on both sides: we enlarged the Java evaluation, and we added a full statistical analysis.

**(1) Larger test sets.** The two added Java benchmarks bring the Java evaluation from 67 to 186 test tasks from three independent sources; SuFu remains a 58-task test set by construction, and we now state its role explicitly.

**(2) Intervals and paired tests.** New App. D (p. 44) reports 95% intervals and exact paired tests for every comparison; the outcomes are:

- **220M, SuFu (58 tasks):** all four differences significant (pass@1  $p = 3.86 \times 10^{-2}$ , pass@10  $p = 2.35 \times 10^{-3}$ , FSP  $p = 7.20 \times 10^{-3}$ , CER  $p = 6.94 \times 10^{-18}$ ).
- **220M, Java (67 tasks):** only CER is significant ( $p = 6.98 \times 10^{-11}$ ); the pass-rate tests are underpowered (one and five extra tasks leave only 11/19/24 discordant pairs), and the appendix explicitly flags them as underpowered.
- **2B, Java pooled (186 tasks):** all four differences significant (pass@1  $p = 2.09 \times 10^{-3}$ , pass@10  $p = 5.36 \times 10^{-4}$ , FSP  $p = 7.65 \times 10^{-4}$ , CER  $p = 1.25 \times 10^{-19}$ ); this analysis carries the statistical conclusion.
- **2B, SuFu (58 tasks):** the compilation-error difference is significant ( $p = 6.94 \times 10^{-18}$ , TyFlow-2B has zero compilation errors on all 313 rendered candidates, so every one of the 58 tasks favors TyFlow); pass@1, pass@10, and FSP favor TyFlow in the same direction as at 220M but are not significant at  $n = 58$ , for the same underpowering reason as the 220M Java block.

Across all comparisons the differences point in the same direction; where the evidence is thinner (220M Java and 2B SuFu), Appendix D states the power limitation and points to the analyses that reach significance.

**(3) One row pair follows the re-trained results.** As noted at the end of this letter, the submitted results come from 2025 runs whose checkpoints are not available here, so all models were re-trained and re-evaluated. This affects the 2B SuFu pair: the re-trained baseline is 25.86/37.93/6.66/71.21 and TyFlow-2B is 36.21/48.28/5.53/0.00, again better on all four metrics. Both rows are backed by preserved per-task arrays.

### Changes made.

- **Added** App. D (p. 44): 95% intervals and exact paired tests at the 220M and 2B scales (Tables 9–10), including the pooled 186-task Java analysis.

- **Added** Sec. 6.1.3 (p. 24): the convention that a task with no correct candidate contributes the candidate-list length to FSP.
- **Revised** Table 1 (p. 26): the SuFu 2B rows, re-verified against preserved per-task records.
- **Revised** Sec. 6.2.1 (p. 25): the pass@10 improvement range (7.46–20.69 points).

#### Revised manuscript, App. D (p. 44)

We pair every CodeT5-220M baseline run with the matching TyFlow-220M run on the same tasks, and compute exact two-sided McNemar tests on pass@1 / pass@10 and exact two-sided sign tests on FSP and on per-task compilation-error fractions (CER). pass@1 and pass@10 report Wilson 95% intervals over tasks, CER over candidates, and FSP a  $t$ -interval over per-task ranks [ . . . ] with ten candidates per task. In the SuFu block all four paired differences favor TyFlow at the 0.05 significance level. In the Java block only CER is significant at  $n = 67$ , for two reasons: the pass-rate improvements themselves are modest at this scale (one additional task at pass@1, five at pass@10), and a 67-task test set yields few discordant pairs (11, 19, and 24 for pass@1, pass@10, and FSP), so the exact tests are underpowered; pooling the three Java benchmarks at the 2B scale restores significance for all four metrics.

#### Revised manuscript, App. D (p. 44)

The three Java benchmarks (MBJP, HumanEval-Java, TransCoder-GFG) all retain per-task records, so we pool them into one combined 186-task analysis rather than reporting each benchmark separately; the same tests and intervals as at the 220M scale apply. In the combined analysis all four paired differences favor TyFlow at the 0.05 significance level.

#### Revised manuscript, App. D (p. 44)

On SuFu only the compilation-error difference is significant at the 2B scale: TyFlow-2B has zero compilation errors on all 313 rendered candidates, so every one of the 58 tasks favors TyFlow in the exact sign test. The pass@1, pass@10, and FSP differences favor TyFlow in the same direction as at the 220M scale but are not significant at  $n = 58$ : the gains are six solved tasks at both pass@1 and pass@10, which leaves few discordant pairs (8 vs. 14 for pass@1, 9 vs. 15 for pass@10) and 30 nonzero FSP pairs (18 better, 12 worse), so the exact tests are underpowered, mirroring the 220M Java block.

#### Comment 1.2.

*On Java, TyFlow-220M improves pass@1 from 10.45% to only 11.94%, less than 1.5 percentage points absolute. While pass@10 improves more substantially, the pass@1 gain is marginal, given the test set size. The paper should discuss why Java gains are smaller.*

[Weaknesses:] *The Java results are modest, and the subset restriction introduces potential selection bias.*

#### Response.

Thanks. We agree that this gain is not convincing on its own. We made three changes: we explain the mechanism, we extend the Java evidence well beyond the 220M figure quoted here, and we address the selection-bias concern directly.

**(1) Why the 220M Java pass@1 gain is small.** Type constraints cannot make a semantically wrong program right; what they remove is uncompileable output, which shows up in pass@10 (20.90% → 28.36%) and CER (35.52% → 1.53%) rather than in pass@1. App. C shows why the top-1 candidate can still be semantically wrong.

**(2) The effect is much larger at 2B and on the added benchmarks.** At the 2B scale, pass@1 rises 13.43% → 25.37% (MBJP), 31.25% → 50.00% (HumanEval-Java), and 19.42% → 30.10% (TransCoder-GFG), with CER falling to 0.45%/1.30%/2.75%. The 220M Java figure is therefore best read as the low-resource end of a scale trend, which the revision now shows.

**(3) Stronger baselines, not weaker ones.** The added benchmarks' baselines were deliberately strengthened (learning rate  $10^{-5}$ , 30 passes, holdout-selected checkpoints): this raised HumanEval-Java from 12.50% to 31.25% pass@1 and TransCoder-GFG from 13.59% to 19.42%, so the TyFlow gains are measured against stronger baselines; the pooled 186-task differences remain significant (Comment 1.1).

**(4) Selection bias.** The Java-subset toolchain admits the tasks it supports (approximately 78% of MBJP). The revision states this admission filter explicitly in App. B and mitigates it with the two additional benchmarks drawn from independent sources; no claim in the paper now rests on the MBJP subset alone.

### Changes made.

- **Added** Sec. 6.2.1 (p. 25): a paragraph explaining why the 220M MBJP pass@1 gain is the smallest.
- **Revised** Sec. 6.1.1 (p. 24): the MBJP protocol, now stated as 608 training and 67 held-out test tasks.
- **Added** App. B (p. 39): the statement that the Java-subset admission filter is a potential selection bias.
- **Added** App. B (p. 40): the baseline protocol of the two added benchmarks (learning rate  $10^{-5}$ , 30 passes, holdouts of 32 and 60 tasks, one test opening).
- **Revised** Table 1 (p. 26): the 2B MBJP rows, measured against the strengthened baseline.

#### Revised manuscript, Sec. 6.2.1 (p. 25)

The 220M MBJP pass@1 gain is the smallest because there the type constraints mainly remove uncompileable candidates; the benefit shows up in pass@10 and CER instead, consistent with the failure analysis in Appendix C (well-typed but wrong is the dominant residual mode). TyFlow also drops CER to 0.00% on SuFu and 0.45% / 1.30% / 2.75% on the three Java benchmarks; the small residual Java CER comes from static-analysis errors the type system does not capture.

#### Revised manuscript, App. B (p. 40)

The T5Gemma2-2B baselines for the two additional benchmarks are fine-tuned with learning rate  $10^{-5}$  for 30 passes, selecting the checkpoint on validation holdouts drawn from the respective training splits (32 and 60 tasks, never in any training data). Each selected configuration was then

evaluated once on the fixed test split, with the standard beam-10 decoding protocol and no few-shot prompting.

**Revised manuscript, App. B (p. 39)**

The subset admits tasks whose constructs our toolchain supports; this admission filter is a potential selection bias, mitigated by the two additional benchmarks drawn from independent sources.

**Comment 1.3.**

*The paper compares only with fine-tuned CodeT5 and T5Gemma2 variants. The absence of any comparison with modern LLMs is a notable gap. The authors could at least report zero-shot pass rates for state-of-the-art proprietary and open-weight LLMs on the same benchmarks to contextualize the absolute numbers.*

[Weaknesses:] *No comparison with modern decoder-only large-scale code models.*

**Response.**

Thanks for this comment. We added exactly the requested zero-shot comparison, plus a few-shot condition, for three larger open-weight decoder-only models (MiMo-7B, Qwen3-14B, Qwen3-30B-A3B) on all four benchmark families (Table 3, p. 26; protocol in the table note). The comparison contextualizes the absolute numbers in both directions:

- **SuFu (58 tasks).** None of the three models solves a single task zero-shot, and few-shot prompting reaches at most 8/58, whereas TyFlow-2B solves 21/58. The low-resource language is precisely where the type-guided representation delivers its advantage, and where scale alone does not.
- **Java.** The prompted models solve 33–45 of 67 MBJP tasks, 3–14 of 16 HumanEval-Java tasks, and 38–52 of 103 TransCoder-GFG tasks, well above the fine-tuned 2B models (TyFlow-2B: 17/8/31); we report this openly, since on Java large decoder-only models are stronger in absolute terms, which is exactly what the comparison is for.
- **A caveat we flag ourselves.** Few-shot prompting brings little or no gain over zero-shot on Java for these models, which suggests they have seen this language extensively and may have been trained on these benchmarks directly; the revision therefore flags a possible data-leakage risk for the Java comparison.

Relative to its fine-tuned baseline T5Gemma2-2B, TyFlow-2B still improves all four benchmarks, which is the controlled comparison the paper’s claims rest on.

**Changes made.**

- **Added** Table 3 (p. 26): pass@1 of MiMo-7B, Qwen3-14B, and Qwen3-30B-A3B on all four benchmarks, zero-shot and few-shot, with the prompt protocol in the table note.

- **Added** Sec. 6.2.1 (p. 26): the reading of this comparison, including the data-leakage caveat.

#### Revised manuscript, Sec. 6.2.1, setup (p. 26)

To contextualize absolute performance, Tab. 3 reports three larger decoder-only models (MiMo-7B, Qwen3-14B, Qwen3-30B-A3B), prompted with the same task prompt in all conditions, alongside our two fine-tuned 2B encoder-decoder models. Each prompted model produces one greedy candidate per task: the zero-shot condition uses only the task prompt, and the few-shot condition prepends three solved examples for the Java benchmarks and six examples for SuFu.

#### Revised manuscript, Sec. 6.2.1, results (p. 26)

On SuFu the comparison favors TyFlow: none of the three solves a single task zero-shot, and few-shot prompting reaches at most 8 of 58, whereas TyFlow-2B solves 21 [. . .] On Java the conclusion is the opposite: the prompted models solve 33–45 of 67 MBJP tasks, 3–14 of 16 HumanEval-Java tasks, and 38–52 of 103 TransCoder-GFG tasks, well above the fine-tuned 2B models. Moreover, few-shot prompting on Java brings little or no gain over zero-shot for these models, which suggests that they have seen this language extensively and may even have been trained on these benchmarks directly; we therefore flag a possible data-leakage risk for the Java comparison.

#### Comment 1.4.

*The example language ( $\lambda \rightarrow$ ) is extremely simple. While it serves the pedagogical purpose well, readers may reasonably wonder whether the approach scales to languages with subtyping, polymorphism, type inference, overloading, or mutable state. The paper partially addresses this by evaluating on Java, but the overview could better acknowledge the gap between  $\lambda \rightarrow$  and real languages.*

*The restriction to first-order unification deserves further discussion. Higher-order unification arises naturally in languages with higher-order functions and polymorphism. The authors should discuss what happens when this restriction is violated and what classes of type systems are excluded.*

[Weaknesses:] *The restriction to first-order unification and encoder-decoder architectures limits near-term applicability.*

#### Response.

Thanks for these questions. We added scoping text at three levels, and we answer each half of the comment separately.

**(1) The example language.** The overview now states that  $\lambda \rightarrow$  serves for exposition and that the evaluation uses Java and SuFu, which have fuller, though still restricted, type systems.

**(2) First-order unification.** Section 3 now states what the restriction excludes (type systems requiring higher-order unification, which is undecidable in general) and what an extension would need; the quoted passage names this class explicitly.

**(3) Type-system expressiveness and the architecture.** A dedicated Limitations subsection (Sec. 6.3) now covers the omitted Java features, the CHC framing, and what migration would

involve (quoted below). The encoder-decoder dependence is treated in the same subsection and in the response to Comment 2.3. Empirically, the two evaluated settings are a functional language (SuFu, 58 test tasks) and a Java subset admitting about 78% of MBJP (186 Java test tasks in total), and TyFlow improves over its base model in both; the added paragraph states precisely what these settings do not establish.

### Changes made.

- **Added** Sec. 2 (p. 9):  $\lambda \rightarrow$  serves for exposition; the evaluation uses Java and SuFu.
- **Added** Sec. 3 (p. 12): what the first-order restriction excludes (higher-order unification) and what an extension would need.
- **Added** Sec. 6.3, “Type-system expressiveness” (p. 30): the omitted Java features, the CHC framing, and future work on richer type systems.

#### Revised manuscript, Sec. 2 (p. 9)

The  $\lambda \rightarrow$  language above serves for exposition; Sec. 6 evaluates TyFlow on Java and SuFu instances with fuller, though still restricted, type systems.

#### Revised manuscript, Sec. 3 (p. 12)

First-order unification is decidable and admits efficient algorithmic solutions, making it suitable for our synthesis framework. This restriction excludes type systems that require *higher-order unification*, which arises naturally with higher-order functions and polymorphism and is undecidable in general. Extending TyFlow to such systems would need a decidable fragment (e.g., patterns) or a different variable-acquisition mechanism, and our formal results do not automatically carry over.

#### Revised manuscript, Sec. 6.3, “Type-system expressiveness” (p. 30)

Both evaluated type systems are relatively simple compared with the mainstream languages that have evolved over decades, such as C++: the implemented Java subset omits subtyping hierarchies, overload resolution, and mutable-state-related features, and SuFu’s type system targets fusion optimization. [...] The restriction is practical rather than conceptual, and our measurements the concrete typing rules and designing the parser for a richer system make experimentation much harder, and our measurements on SuFu and the Java subset do not establish how the approach scales to such languages. Migrating TyFlow to richer type systems, and even to systems such as linear types that can to some extent guarantee functional properties of the code, is a direction for future work.

### Comment 1.5.

*Completeness says that for any well-typed program, there exists a synthesis derivation tree. But during generation, the LM must find the right sequence of decisions. If the LM makes a wrong choice early, it may be unable to recover. The paper does not discuss the branching factor of the search space or the likelihood of the LM making correct decisions in practice. Beam search partially addresses this, but the theoretical analysis gives no bounds on how many beams are needed. The paper does not quantify the pruning rate in Section 5.3. What fraction of candidate tokens are eliminated at each level? This would help readers understand the practical search space*

*reduction.*

## Response.

Thanks for this criticism. We now measure the constrained decoder instead of only arguing about it, and we state explicitly what the completeness theorem does and does not imply for search.

**(1) Pruning rates.** Instrumenting the decoder on the SuFu tasks (beam 10, 330-step budget) gives: syntactic pruning removes 59.1% of all vocabulary proposals, and type pruning a further 8.2% (type checking is applied only to the high-probability candidates about to enter the beam, which is why its rate is lower). A final, complete type check on finished derivations rejects 21.2% of the completed candidates (113/533).

**(2) Branching factor and beam behavior.** The revision separates two claims that were previously conflated: (i) by construction, pruning removes only ill-formed or ill-typed expansions, so every possibly correct derivation survives; (ii) the residual risk comes from beam search itself, which keeps only a fixed number of candidates per step, so the correct candidate may in theory fall outside the beam and the retained candidates then dead-end. In the instrumented run this never happens: under beam 10 no task dead-ends, and every task retains at least one grammar- and type-valid expansion at every step. This is reported as an empirical observation, not as a general guarantee.

## Changes made.

- **Added** Sec. 6.2.2 (p. 27): “How much the constraints prune”: instrumented rates (59.1% syntactic, 8.2% type, 21.2% final check) and the 330-step budget.
- **Added** Sec. 6.2.2 (p. 27): “Whether the constraints over-prune”: the constructive argument, the beam-size residual risk, and the zero dead-end observation.

### Revised manuscript, Sec. 6.2.2, “How much the constraints prune” (p. 27)

To measure the pruning strength, we instrument the constrained decoder on the SuFu tasks (beam 10, 330-step budget): syntactic pruning removes 59.1% of all vocabulary proposals, and type pruning a further 8.2%. The lower rate is expected: type checking is more expensive, so it is applied only to the high-probability candidates about to enter the beam. Such candidates are rarely ill-typed, yet pruning them matters, for they would otherwise expand within the beam and crowd out well-typed derivations. Type checking during search is provisional (only completed AST fields are checked); the whole program is checked once when the derivation finishes, and this final check rejects 21.2% of the completed candidates.

### Revised manuscript, Sec. 6.2.2, “Whether the constraints over-prune” (p. 27)

By construction, pruning removes only ill-formed or ill-typed expansions, so every possibly correct derivation survives. The residual risk is beam search itself, which retains only a fixed number of candidates per step: the correct candidate may in theory fall outside the beam, and the retained candidates then dead-end. In our experiments this never occurs: under the beam-10 setting, no task dead-ends, and every task retains at least one grammar- and type-valid expansion at every step.

**Comment 1.6.**

*What do the remaining failing programs look like? Are they semantically wrong but well-typed? Do they fail on edge cases? A qualitative analysis of failure modes would significantly strengthen the paper.*

**Response.**

Thanks for this suggestion. We added a full failure-analysis appendix (App. C, p. 42–43). Every test task of the three Java benchmarks is classified into one of four categories (App. C defines them), and we give aggregate counts plus two representative cases. The numbers answer the reviewer’s question directly:

- **Invalid generation essentially disappears** under TyFlow: the all-invalid category drops from 4/0/2 to 0/0/2 tasks (MBJP, HumanEval-Java, TransCoder-GFG), and the two residual TransCoder-GFG cases are documented budget-exhaustion (fail-closed) cases rather than ill-formed output.
- **The dominant residual mode is semantic**: 38/50 unsolved MBJP, 7/8 unsolved HumanEval-Java, and 53/72 unsolved TransCoder-GFG tasks have at least one compilable candidate with incorrect behaviour. No typing rule is violated in these cases, so no amount of type-level pruning can reject them, and this is exactly the boundary of what the proposed constraints guarantee, and the revision states it as such.
- **Ranking failures persist** (12–17 tasks on MBJP and TransCoder-GFG): the constrained search often finds a correct derivation without ranking it first, a property of beam scoring rather than of the representation.

**Changes made.**

- **Added** App. C (p. 42–43): the four-category taxonomy, Table 8 with counts for the three Java benchmarks, and two representative cases.
- **Added** Sec. 6.2.1 (p. 25): the sentence linking the 220M MBJP result to the taxonomy.

**Revised manuscript, App. C (p. 42)**

Invalid generation essentially disappears under TyFlow: the all-invalid category drops from 4/0/2 to 0/0/2 tasks, the residual GFG cases being budget exhaustion rather than ill-formed output. Ranking failures persist (12–17 tasks on MBJP and TransCoder-GFG): the constrained search often finds a correct derivation but does not rank it first, a property of beam scoring rather than the representation. The dominant residual mode is well-typed but semantically wrong code: 38/50 unsolved MBJP, 7/8 unsolved HumanEval-Java, and 53/72 unsolved GFG tasks have at least one compilable candidate with incorrect behaviour; this is exactly the boundary of what type-level constraints can guarantee.

**Revised manuscript, App. C.2, representative case (p. 43)**

The candidate has the right shape—a loop over indices with a modulo test—but the body never inspects `nums.get(i)` and never assigns `true`; the method returns `false` for every input, while the tests expect `true` for two of the three cases. No typing rule is violated, so no amount of type-level pruning can reject this candidate: the failure is purely semantic.

## Response to Reviewer 2

We thank the reviewer for the positive assessment of the theoretical foundation, the framework design, and the ablation study.

### Comment 2.1.

*The evaluation uses only a limited number of SuFu test tasks and Java test tasks. These are very small test sets, and no confidence intervals or statistical significance tests are reported. In addition, the SuFu language is highly niche. While this is positioned as a “low-resource” evaluation, it’s unclear how findings generalize beyond this specific setting. There is no evaluation on standard, widely-used benchmarks. While the approach requires language-specific tooling (type checker, parser), the absence of comparison on common benchmarks could limit the paper’s impact and make it harder to contextualize the results.*

[Action item (Major):] *Report confidence intervals or evaluate over multiple random train/test splits to establish statistical stability given the small test sets. Consider adding at least one more widely-used benchmark to help contextualize the results.*

### Response.

Thanks. We did both things the action item asks for, and we state SuFu’s role explicitly rather than claiming generality from it.

**(1) Two widely-used benchmarks.** HumanEval-Java and TransCoder-GFG are added with fixed splits of 146/16 and 414/103 tasks (the quoted passage describes them); the Java evaluation grows from 67 to 186 test tasks across three independent sources. We did not add multiple random splits (the SuFu and added Java splits are fixed by construction); instead we report the intervals and paired tests of the first option.

**(2) Statistical stability.** Rather than reporting point estimates on small sets, App. D reports 95% intervals and exact paired tests for every comparison, at both scales, with the three Java benchmarks pooled into a 186-task analysis: on the pooled set all four metrics are significant, and the appendix identifies explicitly where the evidence remains underpowered (220M Java, 2B SuFu) instead of implying significance (Comment 1.1).

**(3) SuFu’s role.** SuFu stays in the evaluation as the low-resource case, and the revision says so: it is the setting in which the type-guided representation helps most and in which larger decoder-only models fail entirely zero-shot (Comment 1.3), while the Java results provide the

generalization evidence on widely used benchmarks.

### Changes made.

- **Added** Sec. 6.1.1 (p. 24): HumanEval-Java and TransCoder-GFG with their fixed splits (146/16 and 414/103).
- **Added** App. B (p. 39–40): the construction of the two benchmarks and Table 7.
- **Added** App. D (p. 44): the pooled 186-task Java analysis.

#### Revised manuscript, Sec. 6.1.1 (p. 24)

HumanEval-Java and TransCoder-GFG are two additional Java benchmarks that broaden the evaluation beyond MBJP. HumanEval-Java adapts the HumanEval problems to Java, and TransCoder-GFG collects the Java programs released with TransCoder; both follow the task format of MBJP. HumanEval-Java uses a fixed split of 146 training and 16 test tasks, and TransCoder-GFG uses a fixed split of 414 training and 103 test tasks.

#### Revised manuscript, App. B (p. 40, Table 7)

HumanEval-Java and TransCoder-GFG problems are short single-method snippets (median 12 LoC, 21–35-word prompts), whereas MBJP is the densest and most example-rich benchmark (median 27 LoC, prompt with three executable example cases and a gold solution about twice as long as the additional benchmarks).

### Comment 2.2.

*The paper does not discuss how the synthesis derivation tree scales with program complexity. Real-world programs could have deep type derivation trees with polymorphism, subtyping, type inference, and overloading resolution, but the Java subset avoids most of these. It is also unclear what runtime overhead the synthesis system imposes during inference, as the dynamic re-encoding at each step and the unification/constraint-checking machinery introduce costs that are never quantified.*

[Action item (Major):] *Report runtime inference time comparisons between TyFlow and the baselines. Discuss, even qualitatively, how the synthesis system would scale to richer type features such as polymorphism with variance, subtyping hierarchies, or ownership types. Report how frequently beam search exhausts all candidates due to pruning and what the fallback behavior is.*

### Response.

Thanks. We answer all three parts of the action item.

**(1) Runtime overhead, measured.** RQ2 now times the same 220M model in the four component configurations, reporting per-step ( $31.1 \rightarrow 32.4 \rightarrow 43.7 \rightarrow 59.6$  ms) and per-task totals ( $-6.38\%$  /  $+94.00\%$  /  $+173.88\%$ ,  $2.74\times$  the unconstrained base); the quoted paragraph explains where each cost comes from and why syntactic pruning even lowers the per-task total. We do not report wall-clock comparisons against the external methods, since their cost accounting (sampling budgets and engine queries) is not commensurable with ours.

**(2) Scaling to richer type features.** Discussed in the Limitations subsection: what the implemented subset omits (subtyping hierarchies, overload resolution, mutable state), why the CHC formulation is not conceptually limited to it, and, stated explicitly, that our measurements on SuFu and the Java subset do not establish how the approach scales to richer systems. We keep only what the measurements can support.

**(3) Beam exhaustion and fallback.** The instrumented run reports zero tasks that dead-end under beam 10 (every task retains at least one grammar- and type-valid expansion at every decoding step), and the paper states the residual risk plainly: beam search keeps a fixed number of candidates, so the correct candidate may in theory fall outside the beam. The budget-exhaustion behavior that does occur is documented for what it is: the two TransCoder-GFG all-invalid tasks in App. C are budget-exhaustion (fail-closed) cases, counted as failures.

### Changes made.

- **Added** Sec. 6.2.2 (p. 27): the runtime measurements and the Time / ms-per-step columns of Table 4.
- **Added** Sec. 6.2.2 (p. 27): the beam-exhaustion observation (no task dead-ends under beam 10).
- **Added** Sec. 6.3 (p. 29–30): the limitations on type-system expressiveness.
- **Removed:** the earlier qualitative “scaling behavior” paragraph, which asserted deeper trees without evidence.

#### Revised manuscript, Sec. 6.2.2, “Runtime cost” (p. 27)

Each component increases the per-step cost ( $31.1 \rightarrow 32.4 \rightarrow 43.7 \rightarrow 59.6$  ms). Syntactic pruning adds little (roughly +4%); type pruning and the dynamic typing context cost considerably more, as the former invokes the type checker at every step and the latter additionally re-encodes the evolving synthesis goal. The per-task totals move by  $-6.38\%$  /  $+94.00\%$  /  $+173.88\%$  ( $2.74\times$  base). [...] ]

#### Revised manuscript, Sec. 6.2.2 (p. 27)

In our experiments this never occurs: under the beam-10 setting, no task dead-ends, and every task retains at least one grammar- and type-valid expansion at every step.

### Comment 2.3.

*The baselines are CodeT5-220M and T5Gemma2-2B, and both are relatively small encoder-decoder models. There is no comparison with decoder-only models which are much more common in today’s community. Additionally, no model larger than 2B parameters is evaluated. While the paper demonstrates clear improvements over its chosen baselines, it remains an open question whether larger models might already mitigate type errors sufficiently through scale alone. This would contextualize the practical value of the proposed approach.*

[Action item (Major):] *Include at least a zero-shot or few-shot evaluation of a modern decoder-only model on the same benchmarks. Additionally, discuss whether the core ideas could be adapted to*

*decoder-only architectures, even if full integration is left to future work.*

### Response.

Thanks. We did both: the empirical comparison and the architectural discussion.

**(1) The comparison.** Three decoder-only models of 7B, 14B, and 30B parameters are evaluated on all four benchmarks, zero- and few-shot (Table 3; Comment 1.3 quotes the results). The answer to the reviewer’s open question is two-sided: on Java, scale buys absolute performance (33–45 of 67 MBJP tasks vs. 17/67 for TyFlow-2B); on SuFu it does not (0 zero-shot, at most 8/58 few-shot, against 21/58 for TyFlow-2B).

**(2) The architectural discussion.** The Limitations subsection explains what a decoder-only adaptation would involve (quoted below).

#### Revised manuscript, Sec. 6.2.1 (p. 26)

On SuFu the comparison favors TyFlow: none of the three solves a single task zero-shot [. . .]  
On Java the prompted models solve 33–45 of 67 MBJP tasks [. . .] well above the fine-tuned 2B models.

### Changes made.

- **Added** Table 3 (p. 26) and Sec. 6.2.1 (p. 26): the decoder-only comparison.
- **Added** Sec. 6.3, “Dependence on the encoder-decoder architecture” (p. 29): what a decoder-only adaptation requires.

#### Revised manuscript, Sec. 6.3, “Dependence on the encoder-decoder architecture” (p. 29)

The representation itself carries over to a decoder-only model unchanged; what changes is the architecture, and with it the cost of the dynamic typing context. In a decoder-only model, supplying the context would mean prefilling the current typing context after the derivation prefix at every step and removing it again before the next step, so its key-value entries are recomputed at every step and never reused by later steps, and the computation is awkward to schedule. We leave this adaptation to future work.

### Comment 2.4.

*The paper repeatedly emphasizes that CHCs can encode “any Turing-computable function” and that the framework generalizes to “arbitrary logical constraints.” However, the evaluation only covers type correctness for two languages with relatively simple type systems. No evidence is provided for richer constraints. The gap between the theoretical generality and empirical validation is large.*

[Action item (Major):] *Either provide a case study demonstrating a constraint richer than basic type correctness (e.g., ownership, effects, refinement types), or explicitly scope the generality claims to what is empirically demonstrated and acknowledge the gap as a limitation.*

**Response.**

Thanks. We agree that the submitted wording overclaimed. We took the second option and scoped the claim everywhere it appeared (introduction, related work, conclusion), separating what the framework is, what this paper evaluates, and what remains future work. We did not produce the requested case study with an additional constraint family, since such evidence is beyond the scope of this revision; the gap is acknowledged as a limitation instead. The empirical boundary: two languages, four benchmark families, 58 SuFu and 186 Java test tasks, type correctness as the only evaluated constraint family.

**Changes made.**

- **Scope narrowed** Sec. 1 (p. 4), Sec. 7 (p. 30), and Sec. 8 (p. 31): the CHC generality claims now state that the framework is instantiated and evaluated only for type correctness in two languages.

**Revised manuscript, Sec. 1 (p. 4)**

Notably, all proof rules in our system are formulated as constrained Horn clauses. This representation not only subsumes the prior work on syntactic correctness but also accommodates a broader class of Turing-computable specifications. While this paper instantiates and evaluates the framework only for type correctness in two languages, the synthesis construction itself is defined at the CHC level and is in principle applicable to other decidable constraint families; we do not evaluate such extensions empirically.

**Revised manuscript, Sec. 7 (p. 30)**

Grammar-based encoding can be viewed as a special case of our framework when grammar rules are expressed as CHCs. Our implementation and evaluation exercise this generality on type-correctness constraints in two languages; other CHC-representable constraint families remain a formal possibility that we do not evaluate.

**Revised manuscript, Sec. 8 (p. 31)**

Beyond type correctness, TyFlow’s CHC foundation suggests a route toward richer constraint families, such as safety verification and the generation of other structural data; instantiating and evaluating these extensions remains future work.

**Response to Reviewer 3**

We thank the reviewer for the positive assessment of the typing-based extension of grammar-guided generation and of the rigor of the isomorphism between type derivations and synthesis derivations.

**Comment 3.1.**

*Missing comparisons against token-level constrained decoding works such as SynCode and Copiloting the Copilots. Those works also guide LLM generation, but they are better than rejection sampling because they guide the LLM at the token level. To demonstrate the robustness of TyFlow, comparisons against those works are important.*

### Response.

Thanks for this suggestion. RQ3 was rewritten as a five-method comparison on MBJP, with all external methods running on top of one leak-clean, frozen T5Gemma2-2B checkpoint under a shared sampling protocol (temperature 0.8, top- $p$  0.95, 10 candidates per task, the same 67 tasks). This isolates the constraint mechanism rather than the underlying model. Because rejection sampling and repair need diverse draws, the ordinary control uses temperature sampling, so its scores (14.93/23.88) differ slightly from the beam-10 row of Table 2 (13.43/32.84); the TyFlow-2B row is identical in both tables.

The two methods the reviewer names are included and described as they actually work:

- **SynCode** masks, at each decoding step, the tokens that would violate the grammar of our Java subset (adapted; selection rule in the table note). It reduces the compilation-error rate (26.12%  $\rightarrow$  21.34%) but leaves the solved sets unchanged (10/67 and 16/67).
- **Repilot** (*Copiloting the Copilots*) pairs the LM with the Eclipse JDT completion engine: the partial program is maintained as a JDT document and, at each decoding position, the engine is queried for the continuations it can legally complete. In our controlled run its pruning rarely fires, so compilation errors barely move (26.12%  $\rightarrow$  25.97%).

Rejection sampling, the only external method that changes any outcome, adds three solved tasks at pass@10 (19 vs. 16 of 67) and one at pass@1 (11 vs. 10), differences that are not statistically significant (McNemar  $p = 1.00$  and  $p = 0.25$ ). TyFlow-2B, by contrast, solves 17/67 at pass@1 and 29/67 at pass@10 with only 3/670 compilation errors, the best result among all compared methods, consistent with the paper's thesis that internalizing type reasoning changes the model's own output distribution, whereas external mechanisms only filter or repair it.

### Changes made.

- **Rewrote** Sec. 6.2.3 (p. 28): RQ3 is now a five-method comparison under one frozen checkpoint, and Table 5 was replaced accordingly.
- **Added** Sec. 6.2.3 (p. 28): the shared-protocol statement, the reconciliation of the control with Table 2, per-method descriptions, and the paired tests.

### Revised manuscript, Sec. 6.2.3 (p. 28)

On MBJP, four such methods are applied on top of the same frozen T5Gemma2-2B checkpoint selected in RQ1, adding their own pruning or repair mechanisms on top: *rejection sampling*, *SynCode*, *Repilot* (*Copiloting the Copilots*), and *iterative compiler repair*. All methods are evaluated

on the same 67 tasks with 10 candidates each. Because rejection sampling and repair need diverse draws rather than beam output, the ordinary control uses the temperature-sampling decoding protocol (temperature 0.8, top- $p$  0.95), so its scores (14.93/23.88) differ from the beam-10 row of Tab. 2 (13.43/32.84); the TyFlow-2B row is identical in both tables.

**Revised manuscript, Sec. 6.2.3 (p. 28)**

SynCode constrains decoding itself: at each step it masks the tokens that would violate a given grammar, so every output is syntactically well-formed. It is not originally implemented for Java, so we adapt it to the grammar of our Java subset. Repilot pairs the LM with the Eclipse JDT completion engine [...] Repilot’s pruning rarely fires: following the original Repilot implementation, punctuation and keyword tokens are accepted without querying the engine, and the completion list at an incomplete position is permissive, so compilation errors barely move (26.12%  $\rightarrow$  25.97%).

**Revised manuscript, Sec. 6.2.3 (p. 28)**

None of the four external baselines significantly improves functional correctness over the ordinary control: rejection sampling adds three solved tasks at pass@10 (19 vs. 16 of 67) and one at pass@1 (11 vs. 10; McNemar  $p = 1.00$  and  $p = 0.25$ ); SynCode, Repilot, and iterative repair leave the solved sets exactly unchanged (10/67 and 16/67) [...] TyFlow-2B solves 17/67 at pass@1 and 29/67 at pass@10 with 3 of 670 candidates failing to compile, the best of all compared methods. External reliability mechanisms (post-hoc filtering, token-level grammar masking, completion-engine repair, compiler-feedback regeneration) never change the model’s own preference and distribution over output tokens, whereas TyFlow explicitly encodes the typing rules into the model and helps it better learn the program distribution, demonstrating the importance of the Type Explicitness property in Sec. 1.

**Comment 3.2.**

*Missing comparison against iterative refinement approaches: Compared to rejection sampling, iterative refinement approaches regenerate the program with feedback (in this case, the typing failure feedback), which is strictly stronger than rejection sampling as it incorporates typing failure feedback. To demonstrate TyFlow’s robustness, the authors should compare it with this line of work.*

**Response.**

Thanks for this comment. Iterative compiler repair is now one of the five compared methods, with its protocol stated in the revision: `javac` diagnostics are fed back to the model, which regenerates the candidate for at most two rounds; the repair agent is the un-fine-tuned T5Gemma2-2B, since the fine-tuned model never learned to consume diagnostics (using it as its own repairer would misrepresent the method). The measured outcome tests the reviewer’s hypothesis directly: repair improves compilability (compile errors 175  $\rightarrow$  116 of 670 candidates) but adds no solved task: the solved sets remain 10/67 at pass@1 and 16/67 at pass@10, while in the same run rejection sampling adds three solved tasks at pass@10 and one at pass@1, with neither difference significant (McNemar  $p = 1.00$  and  $p = 0.25$ ), so for functional correctness on this benchmark, feedback-based regeneration is not strictly stronger than rejection sampling. The RQ3 discussion explains why: feeding back compile failures changes what compiles but not

the model’s distribution over semantically meaningful programs, whereas TyFlow encodes the typing rules into the model itself. The paired tests for this comparison are in App. D.

### Changes made.

- **Added** Sec. 6.2.3 (p. 28): the repair protocol (javac feedback, at most two rounds, un-fine-tuned agent), its measured outcome, and the closing discussion.

#### Revised manuscript, Sec. 6.2.3 (p. 28)

Iterative compiler repair feeds javac diagnostics back to the model and regenerates the candidate for at most two rounds; the repair agent is the un-fine-tuned T5Gemma2-2B, since the fine-tuned model never learned to consume diagnostics.

#### Revised manuscript, Sec. 6.2.3 (p. 28)

[...] iterative repair repairs 59 candidates’ compilability (compile errors 175→116 of 670) yet leaves the solved sets unchanged.

### Comment 3.3.

*The TyFlow system requires multiple LLM calls to obtain all synthesis decisions. To show the practicality of TyFlow, it is important to provide an analysis of the system’s cost for generating the program.*

### Response.

Thanks for this question. We now measure the cost directly. The derivation is produced in a single constrained beam search (not by repeated sampling of whole programs); its cost is quantified per step and per task for every configuration of the same 220M model: 31.1 → 32.4 → 43.7 → 59.6 ms per decoding step and 5.70 / 5.34 / 11.07 / 15.62 s per task, i.e. −6.38% / +94.00% / +173.88% relative to the unconstrained base (2.74× in the full configuration). Each increment has a stated cause (per-step type checking; re-encoding the evolving synthesis goal). For the compared external methods, the RQ3 setup states each method’s computational budget rather than leaving costs unstated: rejection sampling draws up to 100 candidates per task, iterative repair regenerates at most twice, and SynCode and Repilot decode the same ten candidates with their constraint machinery applied at every decoding step. Because these budgets are accounted differently (candidate draws vs. per-token engine queries vs. per-step type checking), each method’s budget is stated in its own description.

### Changes made.

- **Added** Sec. 6.2.2 (p. 27): per-step and per-task runtime for the four configurations (Table 4).
- **Added** Sec. 6.2.3 (p. 28): the computational budget of each compared external method.

#### Revised manuscript, Sec. 6.2.3 (p. 28)

Rejection sampling repeatedly draws candidates and keeps those that pass Java compilation, for up to ten ten-draw rounds (100 candidates) per task.

**Revised manuscript, Sec. 6.2.2, “Runtime cost” (p. 27)**

Syntactic pruning adds little (roughly +4%); type pruning and the dynamic typing context cost considerably more, as the former invokes the type checker at every step and the latter additionally re-encodes the evolving synthesis goal.

**Comment 3.4.**

*Although TyFlow brings significant improvement on SuFu, its performance boost on Java is marginal (from 10.45% to 11.94%).*

**Response.**

Thanks for this observation. The figure quoted here is the 220M MBJP result, the smallest effect in the paper. The 220M pass@1 gain is small because the type constraints mainly remove uncompileable candidates; the benefit shows up in pass@10 and CER instead (both quoted below), and App. C shows why the top-1 candidate can still be semantically wrong even when it compiles. The Java picture at 2B is much stronger (quoted under Comment 1.2), and those gains are measured against deliberately strengthened baselines, with all four pooled 186-task differences significant (Comment 1.1). The 220M result remains in Table 1 for completeness; the Java conclusions now rest on the 2B results and the pooled analysis.

**Revised manuscript, Sec. 6.2.1 (p. 25)**

TyFlow improves code generation accuracy at both scales and on every benchmark: pass@1 rises from 24.14% to 37.93% (SuFu) and from 10.45% to 11.94% (MBJP) at 220M, and from 25.86% to 36.21% (SuFu), 13.43% to 25.37% (MBJP), 31.25% to 50.00% (HumanEval-Java), and 19.42% to 30.10% (TransCoder-GFG) at 2B; pass@10 improves by 7.46–20.69 points on the same six benchmarks.

**Revised manuscript, Sec. 6.2.1 (p. 25)**

The 220M MBJP pass@1 gain is the smallest because there the type constraints mainly remove uncompileable candidates; the benefit shows up in pass@10 and CER instead, consistent with the failure analysis in Appendix C (well-typed but wrong is the dominant residual mode).

**Changes made.**

- **Added** Sec. 6.2.1 (p. 25): the paragraph explaining why the 220M MBJP gain is the smallest.
- **Revised** Table 1 (p. 26): the 2B Java rows (MBJP, HumanEval-Java, TransCoder-GFG).
- **Added** App. B (p. 40) and App. D (p. 44): the baseline protocol and the pooled significance analysis behind these rows.

**Comment 3.5.**

*The TyFlow system was mainly evaluated on small models. Evaluation on larger models (decoder-only, billions of parameters) is unexplored. The authors should consider extending the evaluation to larger models.*

**Response.**

Thanks for this comment. The evaluation now spans five model scales: CodeT5-220M and T5Gemma2-2B (both encoder-decoder, fine-tuned with TyFlow) and the decoder-only MiMo-7B, Qwen3-14B, and Qwen3-30B-A3B (zero- and few-shot, Table 3, p. 26). The decoder-only models use a separate prompt-based protocol, kept distinct from the trained results. Their results give the direct answer to the question of larger models: on Java they reach clearly higher absolute pass@1 than the fine-tuned 2B models (e.g., 33–45 of 67 MBJP tasks against 17/67 for TyFlow-2B), while on SuFu none of them solves a single task zero-shot and few-shot prompting reaches at most 8 of 58, against 21/58 for TyFlow-2B; scale alone therefore does not substitute for the type-guided representation in the setting the method targets. Whether TyFlow itself can be adapted to decoder-only architectures is discussed in the Limitations (quoted under Comment 2.3), including what the adaptation would cost.

**Revised manuscript, Sec. 6.2.1, Table 3 and its note (p. 26)**

To contextualize absolute performance, Tab. 3 reports three larger decoder-only models (MiMo-7B, Qwen3-14B, Qwen3-30B-A3B), prompted with the same task prompt in all conditions, alongside our two fine-tuned 2B encoder-decoder models. Each prompted model produces one greedy candidate per task [ . . . ] Cell entries are solved tasks at pass@1 for all models; the 0-shot condition uses the same prompt as the two fine-tuned models.

**Changes made.**

- **Added** Table 3 (p. 26): the three decoder-only models on all four benchmarks.
- **Added** Sec. 6.3 (p. 29): the architectural limitation for a decoder-only adaptation.

**Comment 3.6.**

*TyFlow relies strongly on the typing system of the language, but how would it work on a dynamically typed language such as Python? Thus, the author should acknowledge this in the threat to validity to make this paper more robust.*

**Response.**

Thanks for this comment. A dedicated limitations paragraph now states the obstacle: TyFlow needs the type derivation to be computable from the program text, which is what makes training data automatic in the two evaluated languages (290 SuFu programs, 608 Java training tasks); in a dynamically typed language the derivation depends on control flow and runtime information,

so decision sequences cannot be extracted from existing code, the Data Usability property breaks, and adaptation is left to future work.

**Revised manuscript, Sec. 1, “Data Usability” (p. 3)**

The decision sequences of  $S$  and source code should be automatically and bidirectionally convertible, enabling both the extraction of decision sequences from existing programs and the reconstruction of programs from generated decision sequences.

**Changes made.**

- **Added** Sec. 6.3, “Dynamically typed languages” (p. 29): the limitation and why the training-data pipeline does not carry over.

**Revised manuscript, Sec. 6.3, “Dynamically typed languages” (p. 29)**

The current design of TyFlow relies on a type system in which the type derivation of a program can be computed automatically from the program text, so that existing code can be converted into our decision-sequence format. Dynamically typed languages such as Python still have typing rules to a degree, but the type derivation cannot be read off the program text alone, since types depend on control flow and runtime information. [...] Parsing and adapting such languages is left to future work.

### Other Changes beyond the Reviewers’ Requests

- **Re-trained results.** The results in the submitted version were produced in 2025 in a different environment, and those checkpoints and evaluation records are not available on the server used for this revision; every model reported here was therefore re-trained with the same framework and the same code, and all evaluations were re-run. The new run supports the conclusions of the submitted version; the differences are small (most values move by less than two percentage points; the largest is the 2B SuFu row, where the re-measured baseline is 25.86% (submitted: 29.31%) and the reported TyFlow-2B value is 36.21% (submitted: 43.10%); TyFlow still exceeds the baseline on all four metrics); the intervals and paired tests of Appendix D are computed on the new runs, so the significance statements apply to the numbers printed in the tables.
- **Errata.** Algorithm 1 now composes substitutions as  $\sigma_{i-1} \dots \sigma_0 \sigma$ ; the statement after Lemma 2 reads  $P(\theta(\bar{i}))$ ; and the proof of Lemma 4 defines  $\theta_c = \sigma_n \dots \sigma_0 \sigma$ . The method and the theorems are unchanged.
- **FSP definition.** Sec. 6.1.3 (p. 24): a task with no correct candidate contributes the candidate-list length, a convention the submitted text left implicit.
- **RQ3 table.** The SuFu rows of the submitted table had no backing records; they were removed, and RQ3 is now the MBJP comparison of five methods (Sec. 6.2.3, p. 28).
- **RQ4 accounting.** Sec. 6.2.4 (p. 29): the plain baseline rows were added and the token-

length accounting restated (generated sequence in each model’s own vocabulary, excluding comments).

- **Traceability.** All rows reported in this revision are backed by preserved checkpoints and per-task records.

We hope that the revision addresses all concerns and look forward to the reviewers’ feedback.

Sincerely,  
The Authors